GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

通过 llm-d 打造跨负载、跨模态、跨硬件、跨平台的分布式推理平面

陈楚晖

红帽大中华区

首席架构师



CONTENTS

Part 01 项目的核心理念

Part 02 架构组件

Part 03 调度优化策略

Part 04 最新的版本进展

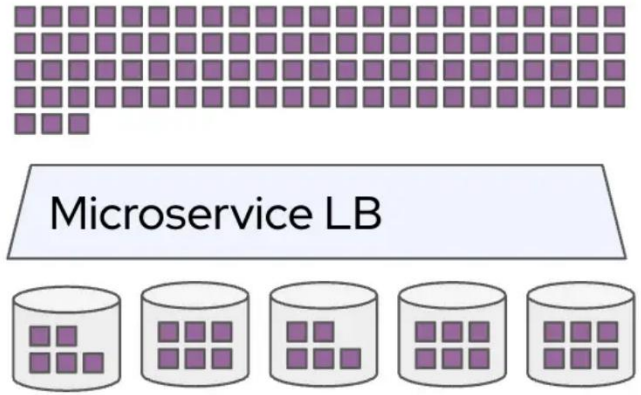


01 项目的核心理念



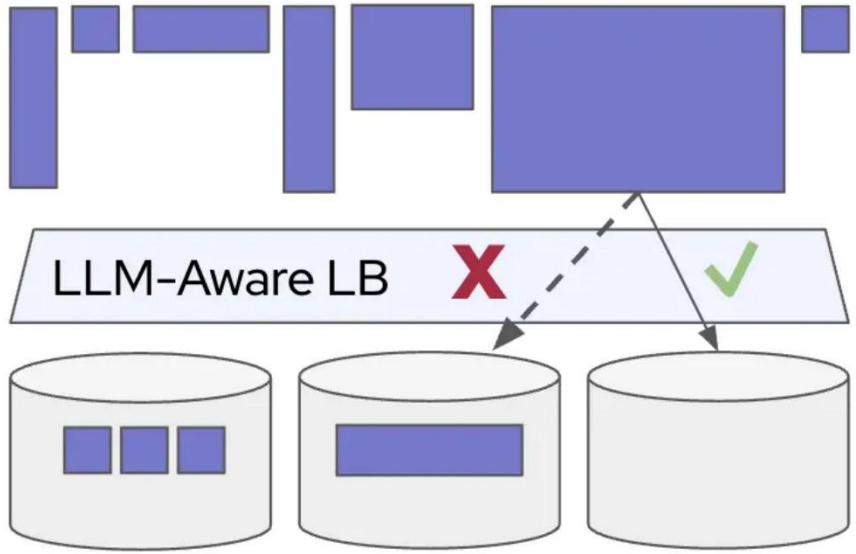
为什么要进行分布式推理?

Modern HTTP requests:
Fast, uniform, cheap



3-4 orders of magnitude more QPS

LLM requests:
Slow, non-uniform, really expensive

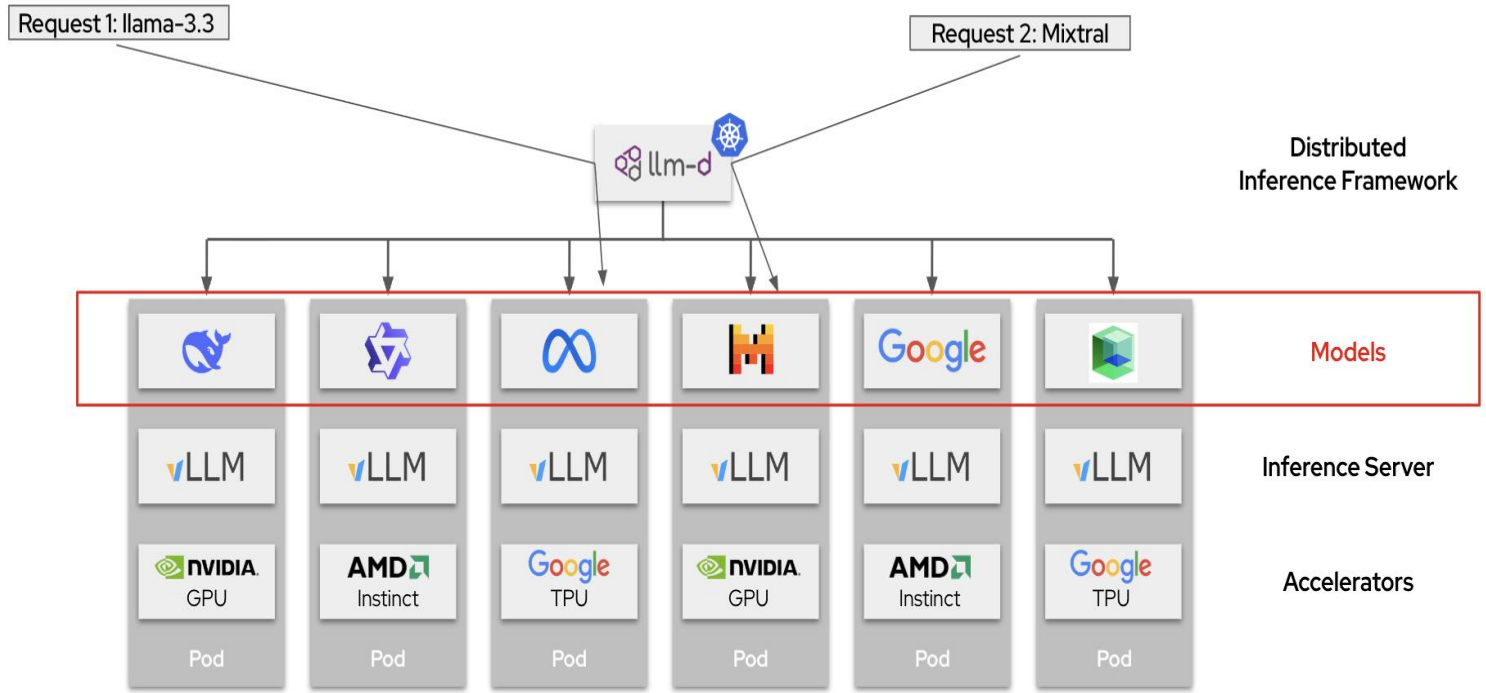


6-9 OoM more expensive per request

llm-d 是什么?

一个用于分布式大语言模型（LLM）推理的开源、Kubernetes 原生框架

一个**在 Kubernetes 上原生运行**的**分布式**大语言模型（LLM）推理开源框架。



- 由 Red Hat、Google、Hugging Face 及众多其他组织联合发起的**开源项目**
- **支持任意环境部署——包括 Kubernetes (k8s) 及非 k8s 环境（新增特性）**，并提供日益广泛的硬件支持
- **分布式且支持插件扩展**的架构
- 提供**引导式部署路径**，展示**先进的调度技术**与**服务模式**

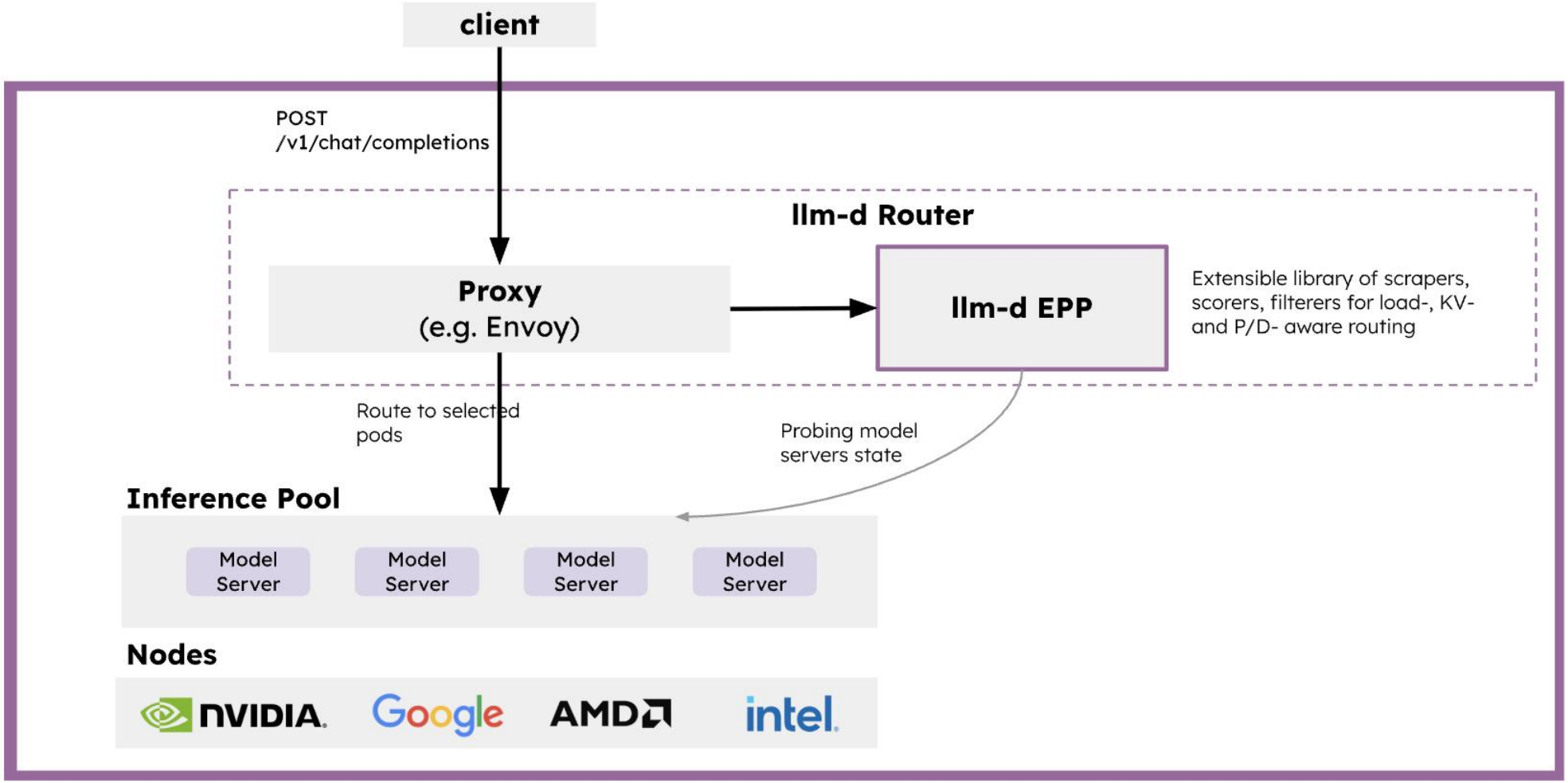
Red Hat IBM Google CoreWeave Hugging Face NVIDIA

02

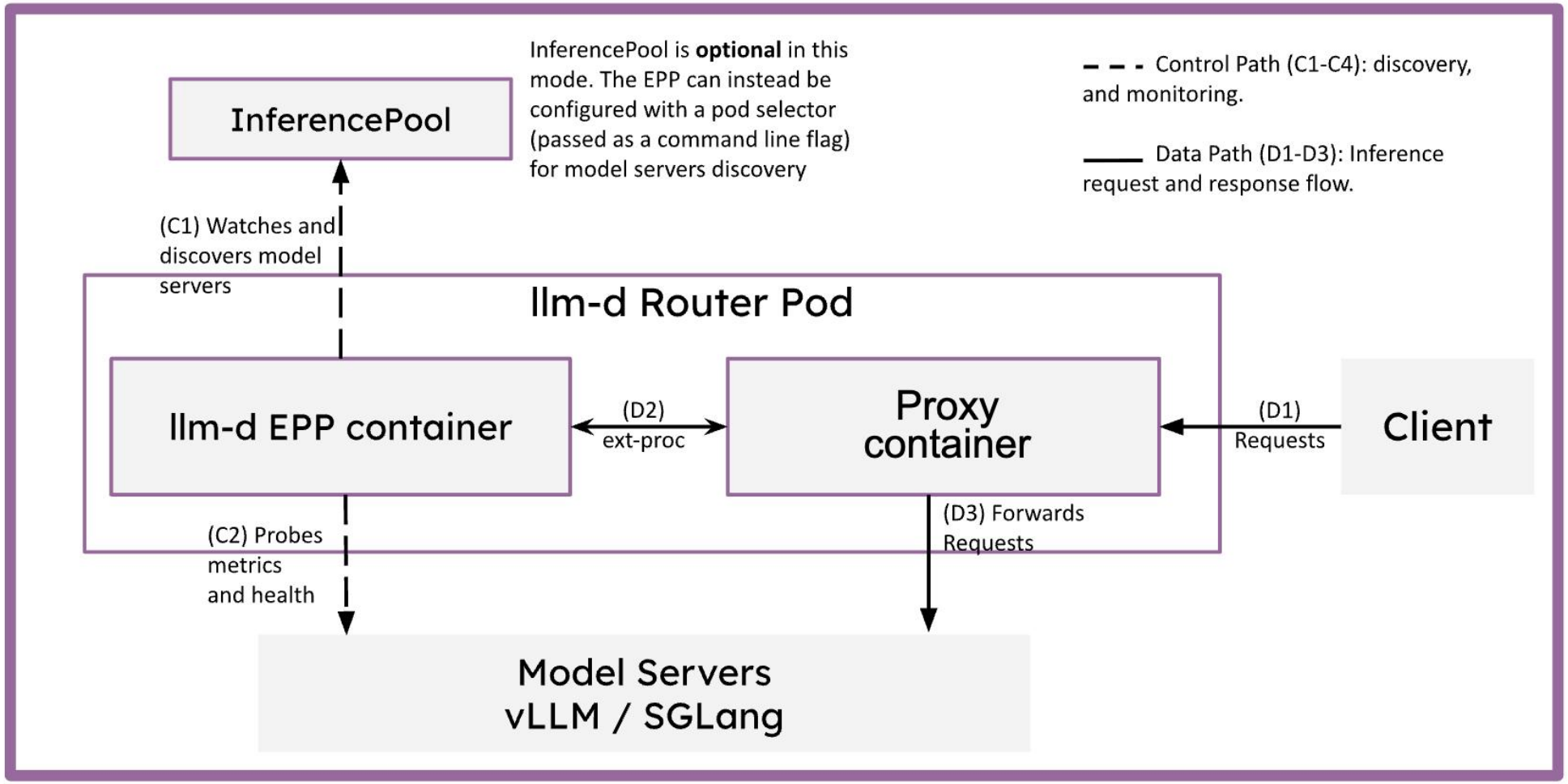
架构组件



llm-d 核心组件：Router, Inference Pool, Model Servers

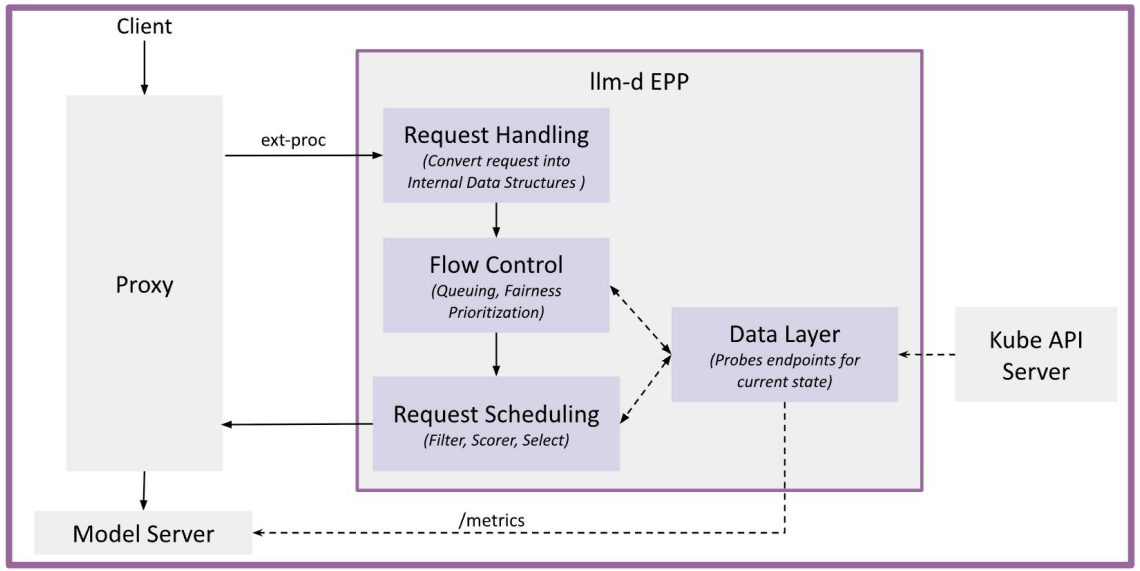


代理层（Proxy）

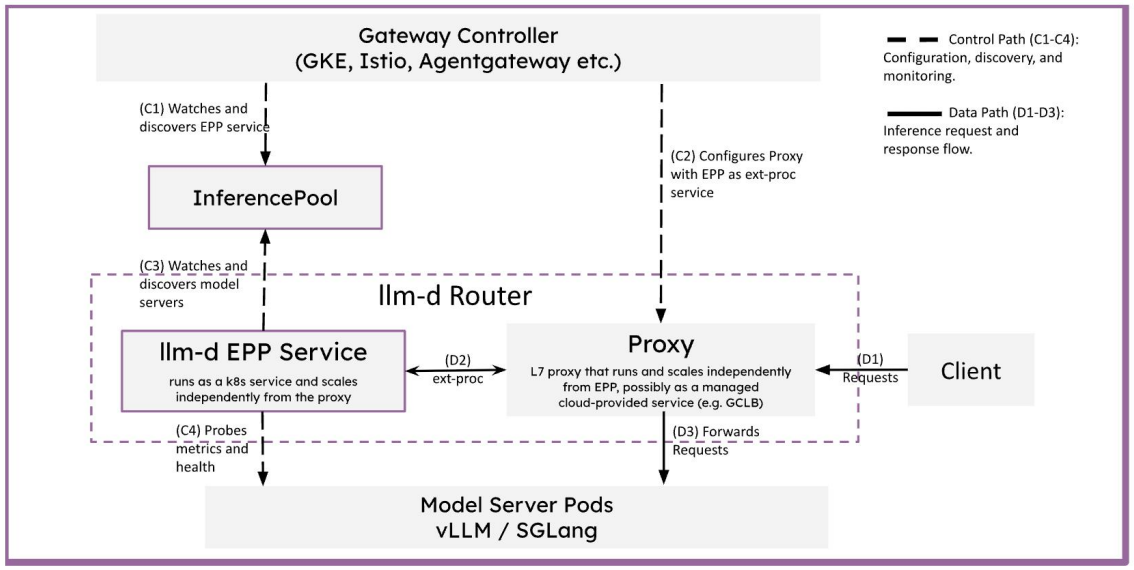

InferencePool
InferencePool is **optional** in this mode. The EPP can instead be configured with a pod selector (passed as a command line flag) for model servers discovery
Control Path (C1-C4): discovery, and monitoring.
Data Path (D1-D3): Inference request and response flow.
(C1) Watches and discovers model servers
llm-d Router Pod
llm-d EPP container
(D2) ext-proc
Proxy container
(D1) Requests
Client
(C2) Probes metrics and health
(D3) Forwards Requests
Model Servers vLLM / SGLang

代理的两种模式 — 独立模式与网关模式

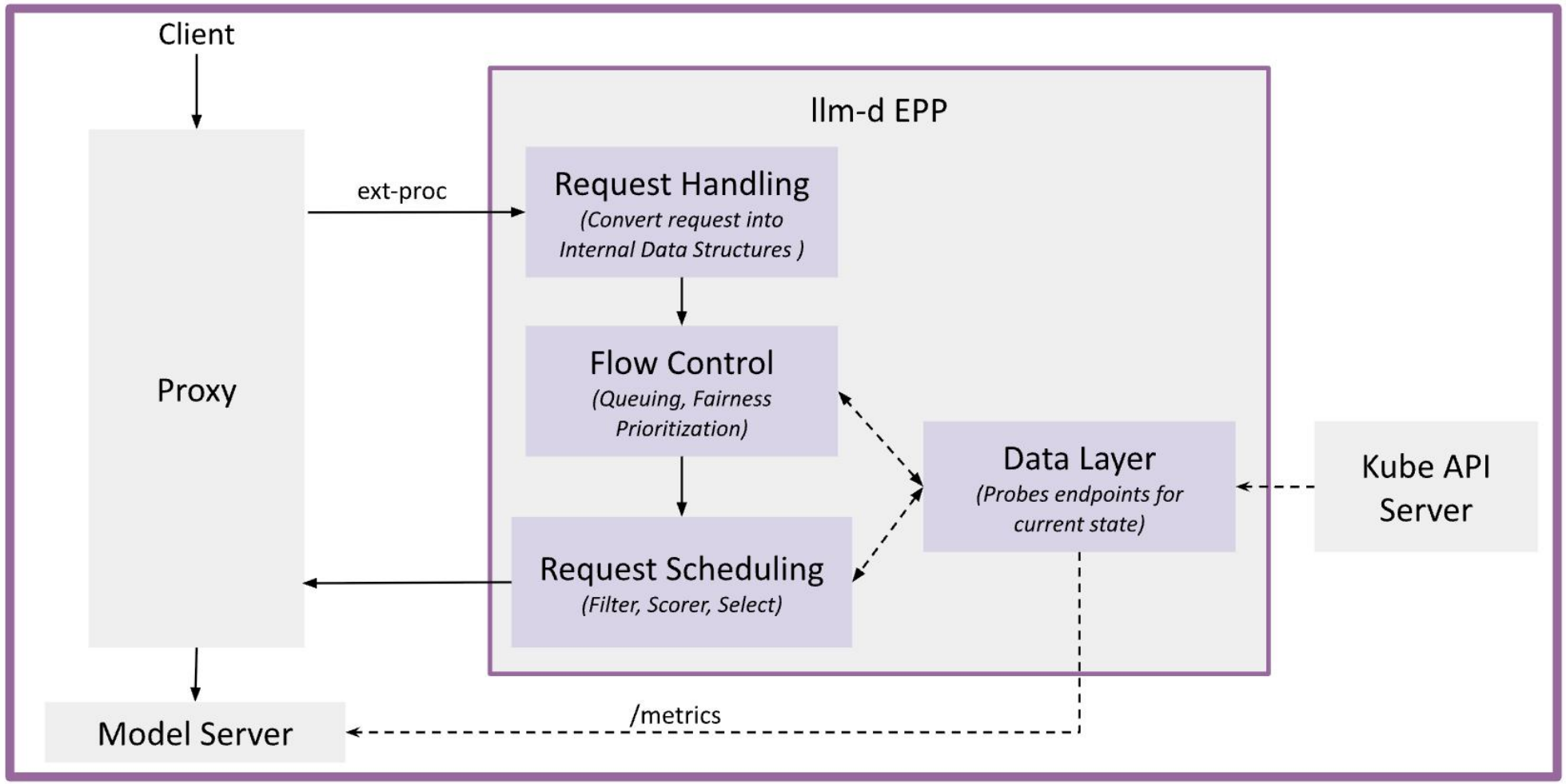
Standalone Mode

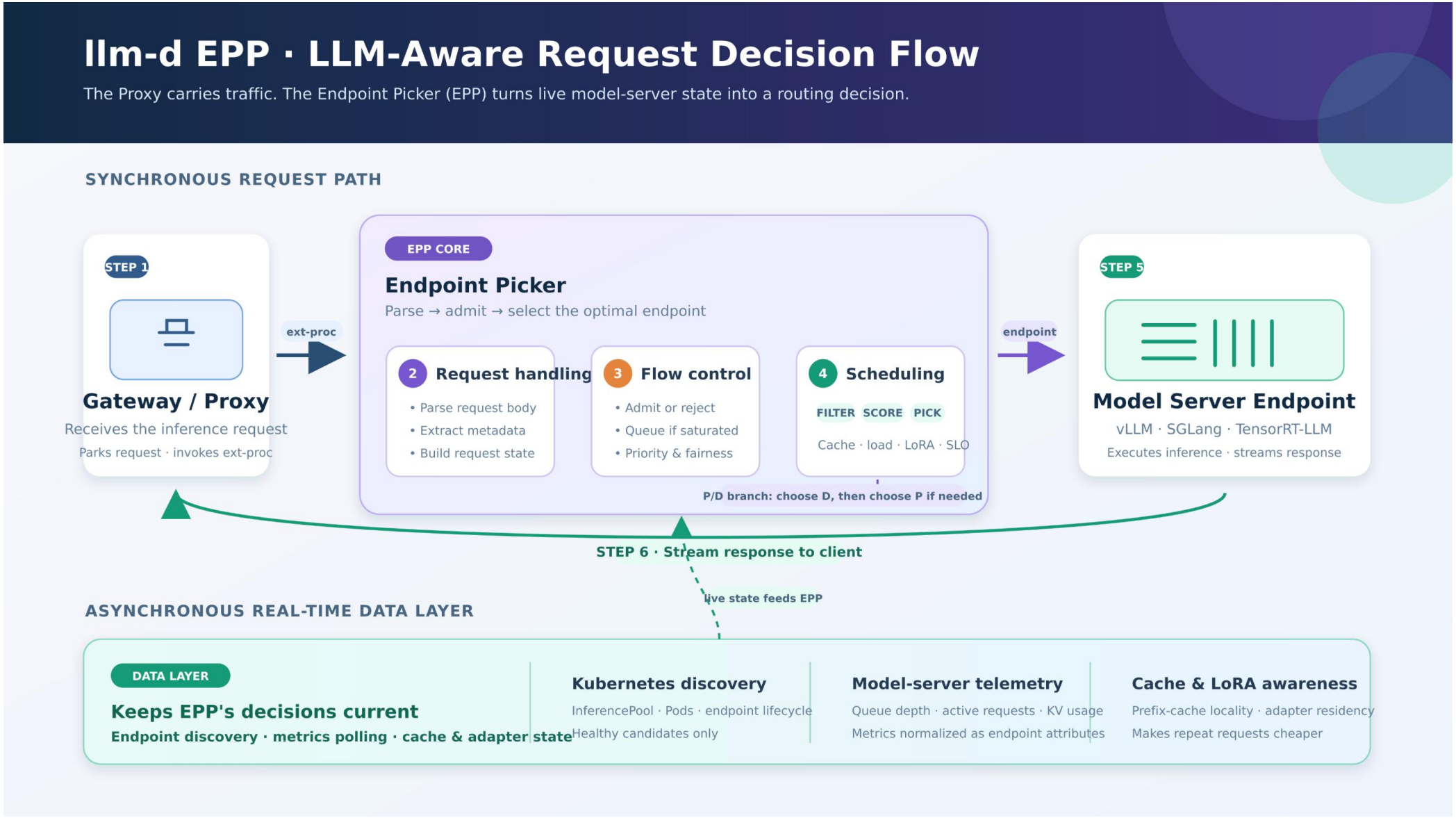


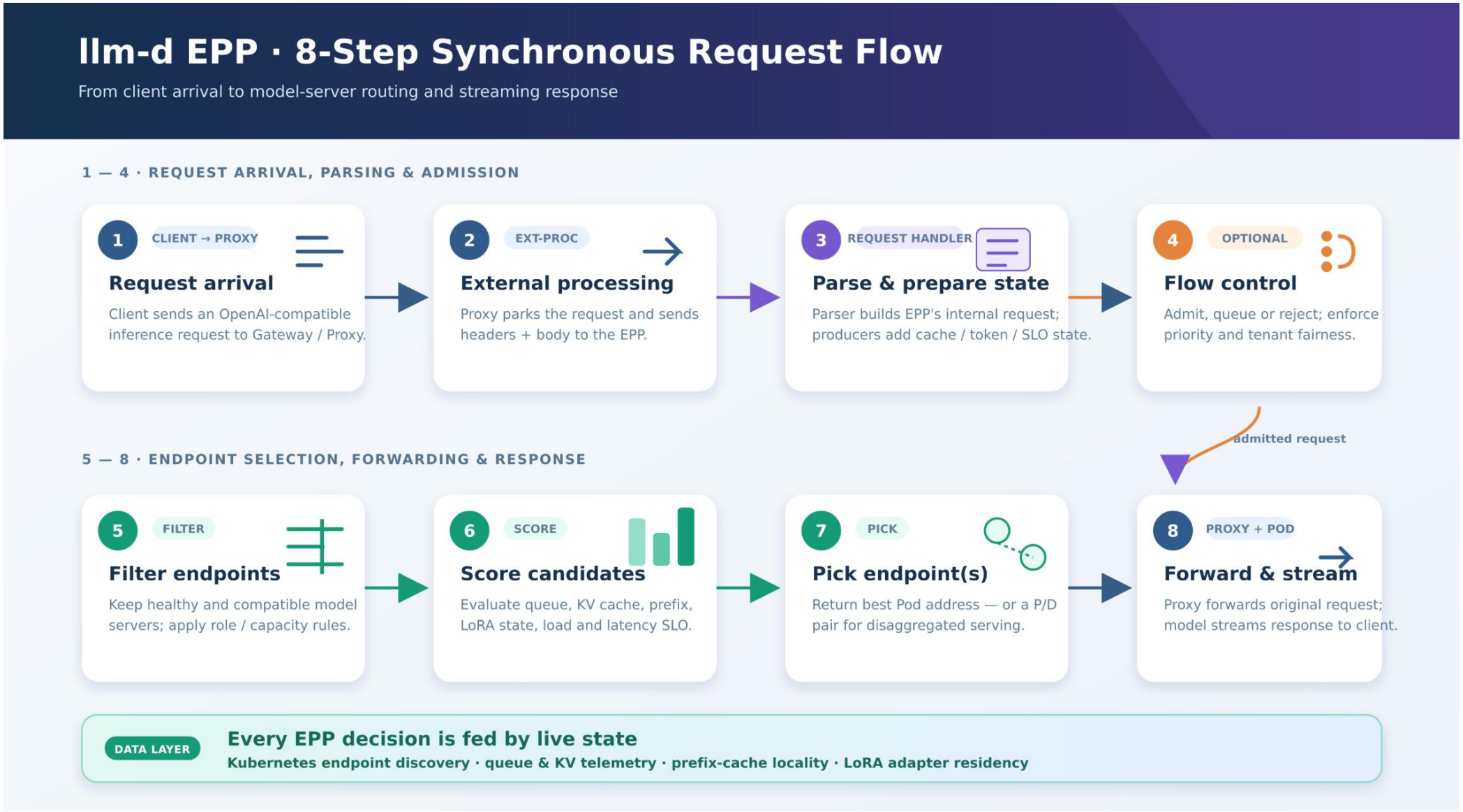
Gateway Mode



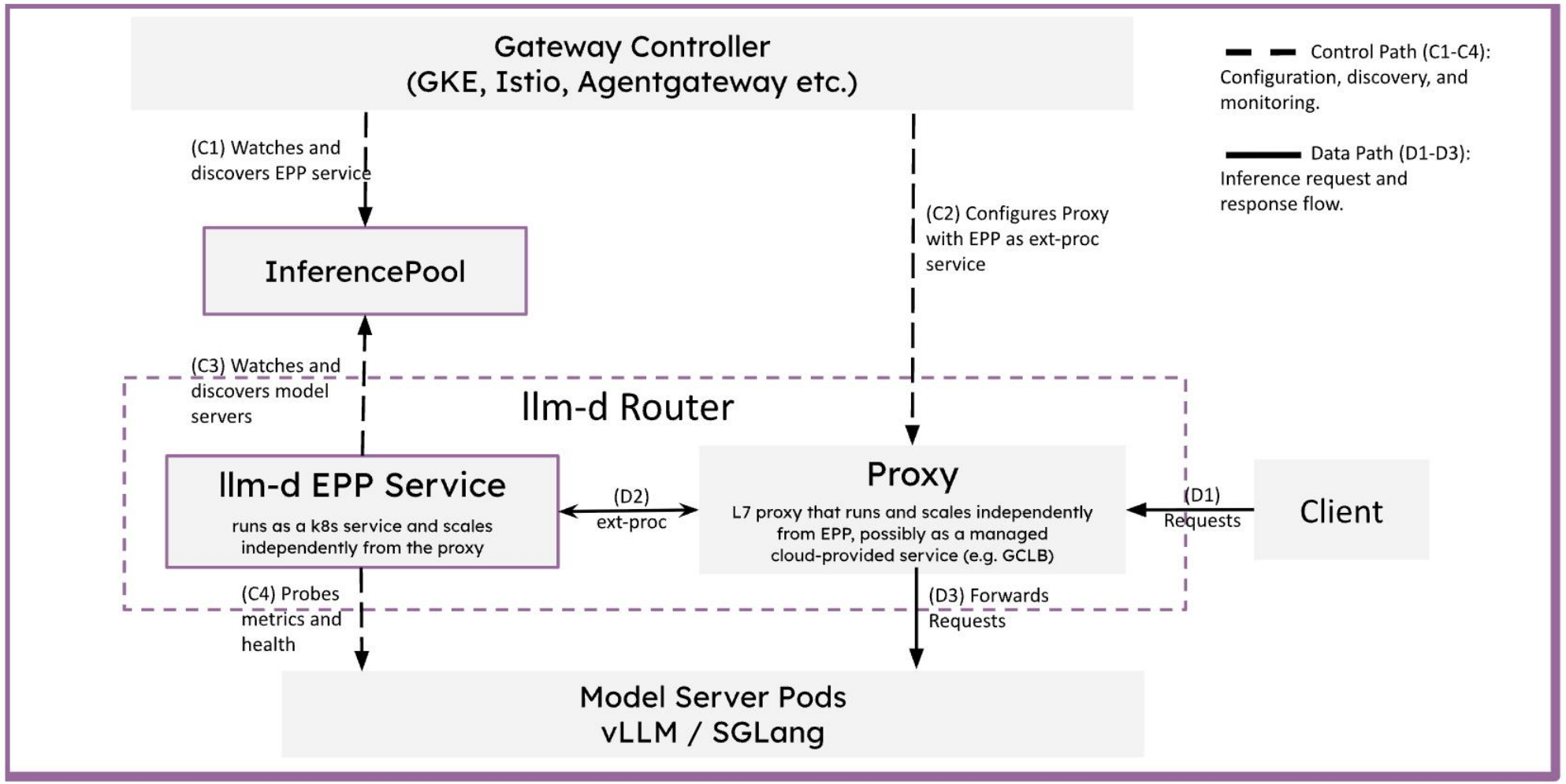
端点选择器（Endpoint Picker，EPP）




llm-d EPP · LLM-Aware Request Decision Flow
The Proxy carries traffic. The Endpoint Picker (EPP) turns live model-server state into a routing decision.
SYNCHRONOUS REQUEST PATH
STEP 1
Gateway / Proxy
Receives the inference request
Parks request · invokes ext-proc
ext-proc
EPP CORE
Endpoint Picker
Parse → admit → select the optimal endpoint
2 Request handling
• Parse request body
• Extract metadata
• Build request state
3 Flow control
• Admit or reject
• Queue if saturated
• Priority & fairness
4 Scheduling
FILTER SCORE PICK
Cache · load · LoRA · SLO
P/D branch: choose D, then choose P if needed
endpoint
STEP 5
Model Server Endpoint
vLLM · SGLang · TensorRT-LLM
Executes inference · streams response
STEP 6 · Stream response to client
live state feeds EPP
ASYNCHRONOUS REAL-TIME DATA LAYER
DATA LAYER
Keeps EPP's decisions current
Endpoint discovery · metrics polling · cache & adapter state
Kubernetes discovery
InferencePool · Pods · endpoint lifecycle
Healthy candidates only
Model-server telemetry
Queue depth · active requests · KV usage
Metrics normalized as endpoint attributes
Cache & LoRA awareness
Prefix-cache locality · adapter residency
Makes repeat requests cheaper


llm-d EPP · 8-Step Synchronous Request Flow
From client arrival to model-server routing and streaming response
1 — 4 · REQUEST ARRIVAL, PARSING & ADMISSION
1
CLIENT → PROXY
Request arrival
Client sends an OpenAI-compatible inference request to Gateway / Proxy.
2
EXT-PROC
External processing
Proxy parks the request and sends headers + body to the EPP.
3
REQUEST HANDLER
Parse & prepare state
Parser builds EPP's internal request; producers add cache / token / SLO state.
4
OPTIONAL
Flow control
Admit, queue or reject; enforce priority and tenant fairness.
admitted request
5 — 8 · ENDPOINT SELECTION, FORWARDING & RESPONSE
5
FILTER
Filter endpoints
Keep healthy and compatible model servers; apply role / capacity rules.
6
SCORE
Score candidates
Evaluate queue, KV cache, prefix, LoRA state, load and latency SLO.
7
PICK
Pick endpoint(s)
Return best Pod address — or a P/D pair for disaggregated serving.
8
PROXY + POD
Forward & stream
Proxy forwards original request; model streams response to client.
DATA LAYER
Every EPP decision is fed by live state
Kubernetes endpoint discovery · queue & KV telemetry · prefix-cache locality · LoRA adapter residency

推理池（InferencePool）



关键职责：

- **提供推理服务：**通过与 OpenAI 兼容的 API（`/v1/completions`、`/v1/chat/completions`）提供推理服务。
- **暴露指标：**暴露（KV Cache 利用率、队列深度、活跃请求数）等指标，供 EPP 进行智能调度。
- **管理 KV Cache：**管理 GPU 显存中的 KV Cache，包括对重复提示词前缀进行前缀缓存。
- **支持并行策略：**支持大模型并行策略，例如张量并行（TP）、数据并行（DP）和专家并行（EP）。



03

调度优化策略



经过验证的最佳实践路径（Well-Lit Paths）

精心设计的端到端指南，涵盖 llm-d 原生推理模式、智能路由算法和运行时流量调控引擎。

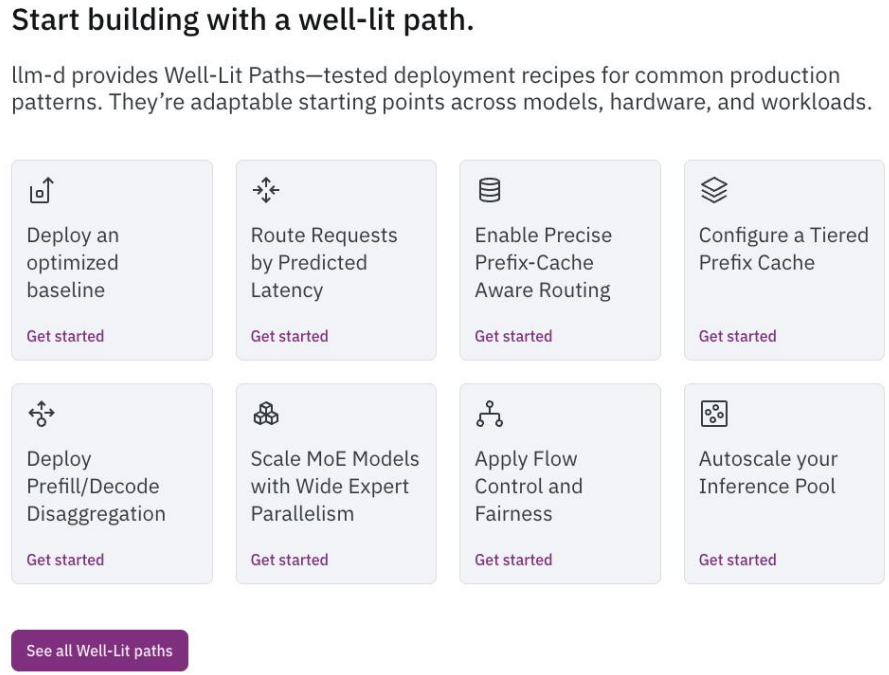
这些指南分为两类原生架构：

基础能力（Foundations）

单项功能特性、复杂路由算法，以及实际的推理执行路径。

工作负载（Workloads）

面向生产环境的一体化部署指南，将各项能力构建模块组合为一个针对特定使用场景调优的横向技术栈。



优化基线（Optimized Baseline）

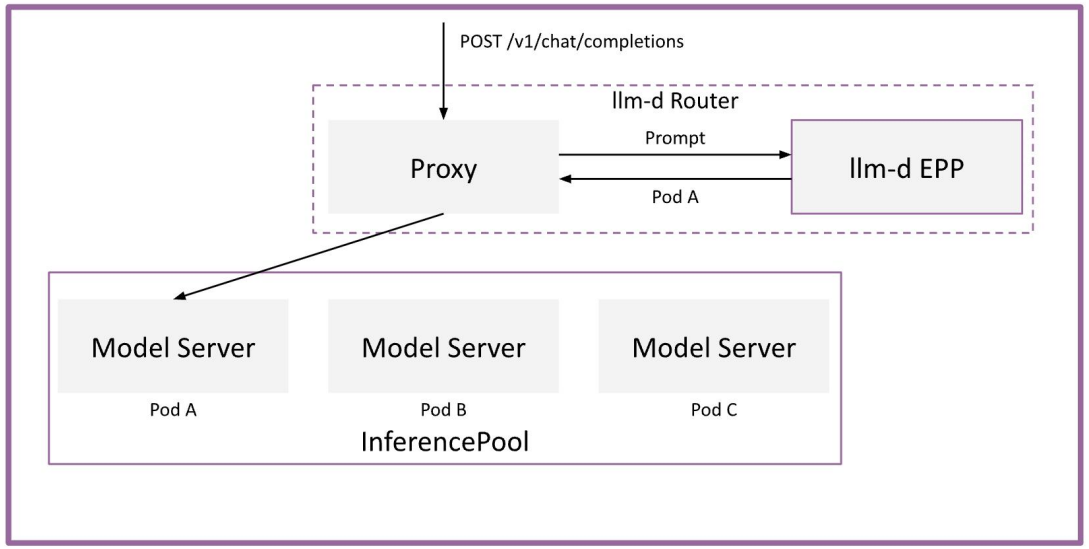
传统的 HTTP 请求具有**速度快**、**形式统一**且**成本低廉**的特点。标准的轮询（round-robin）请求调度策略能很好地平衡负载。

而LLM 请求打破了上述三项假设，具体表现为：

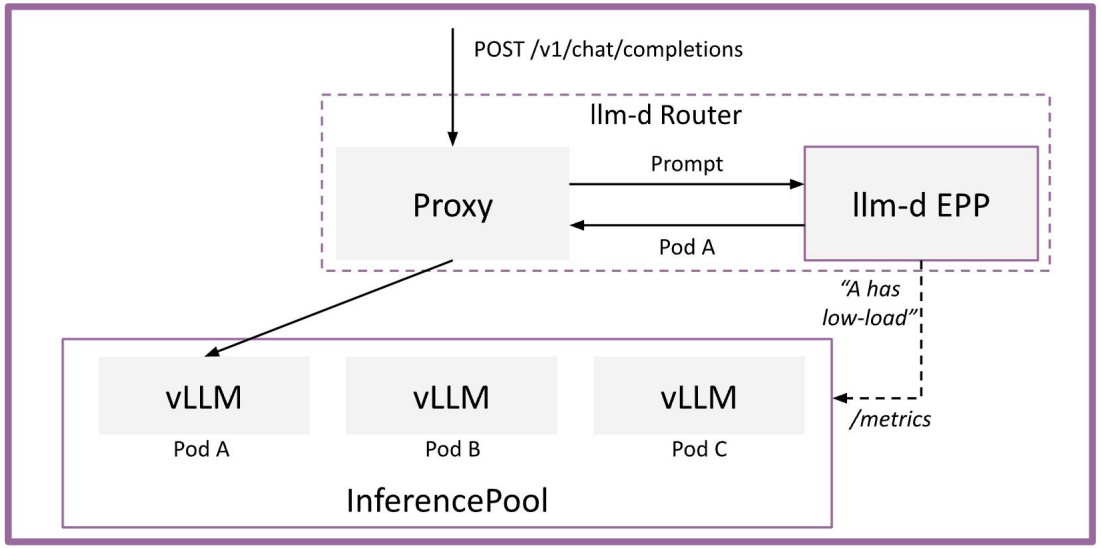
- **多轮（Multi-turn）**：对话和智能体工具调用循环会反复发送相同且不断增长的提示词前缀。
- **缓慢（Slow）**：单个请求生成 Token 的时间可能超过一分钟。
- **非均匀（Non-uniform）**：请求规模差异很大，从数千个推理 Token 到超过 10 万个上下文 Token 不等。

llm-d 路由组件将 LLM 工作负载感知能力引入负载均衡层，综合考量**前缀缓存亲和性**与**服务器负载指标**。

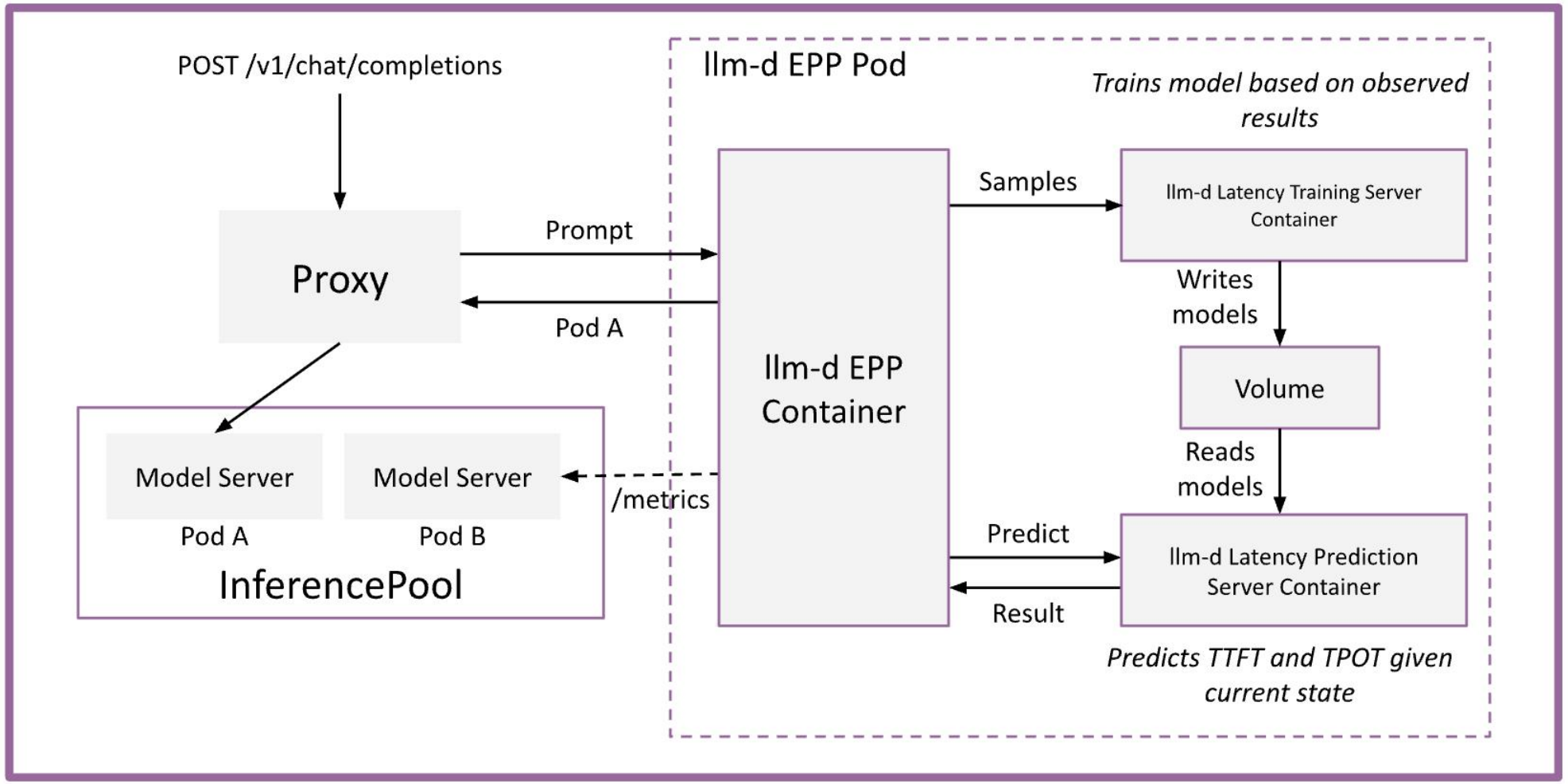
前缀感知调度（Prefix-Aware Scheduling）

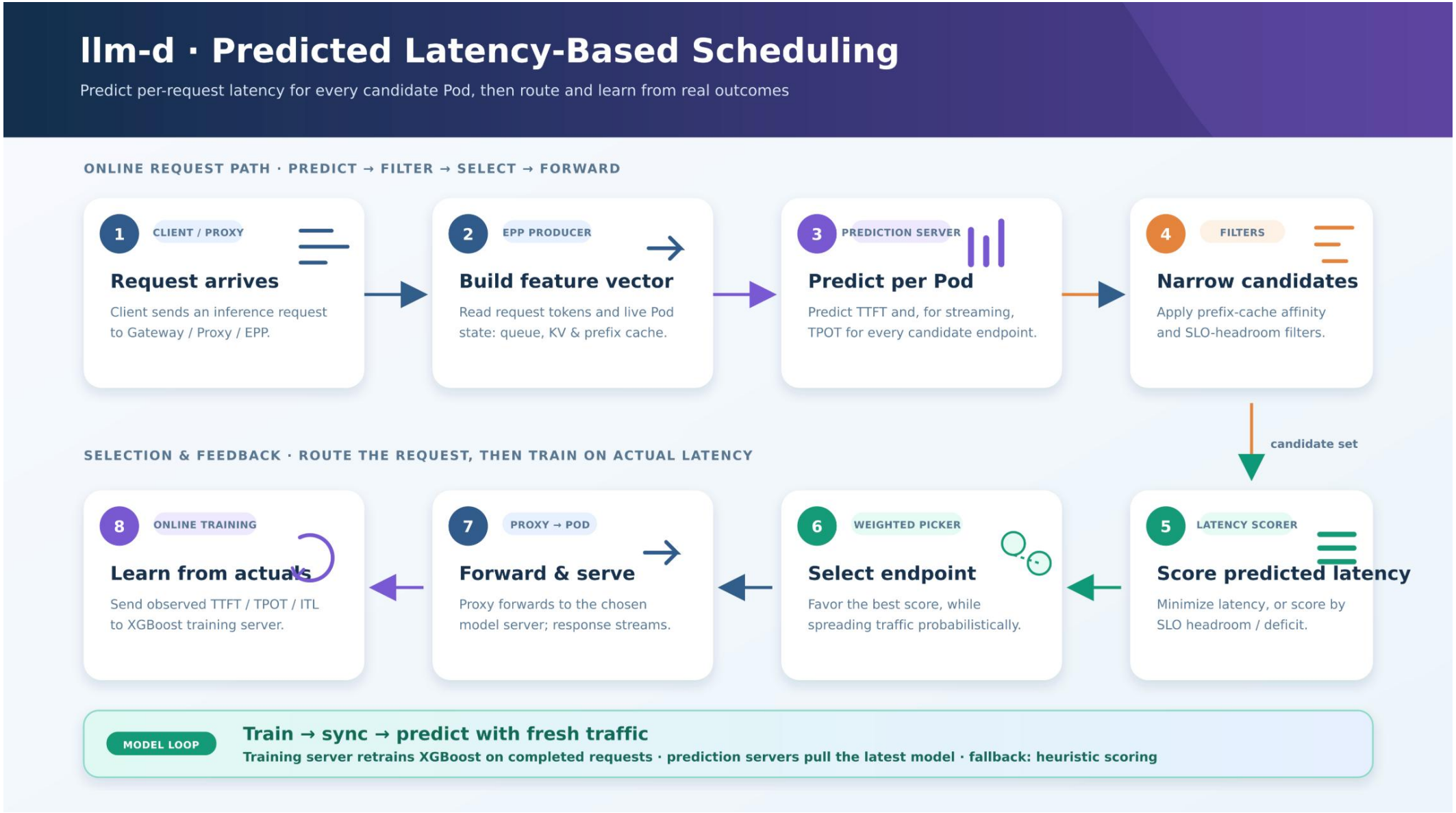


负载感知调度（Load-Aware Scheduling）

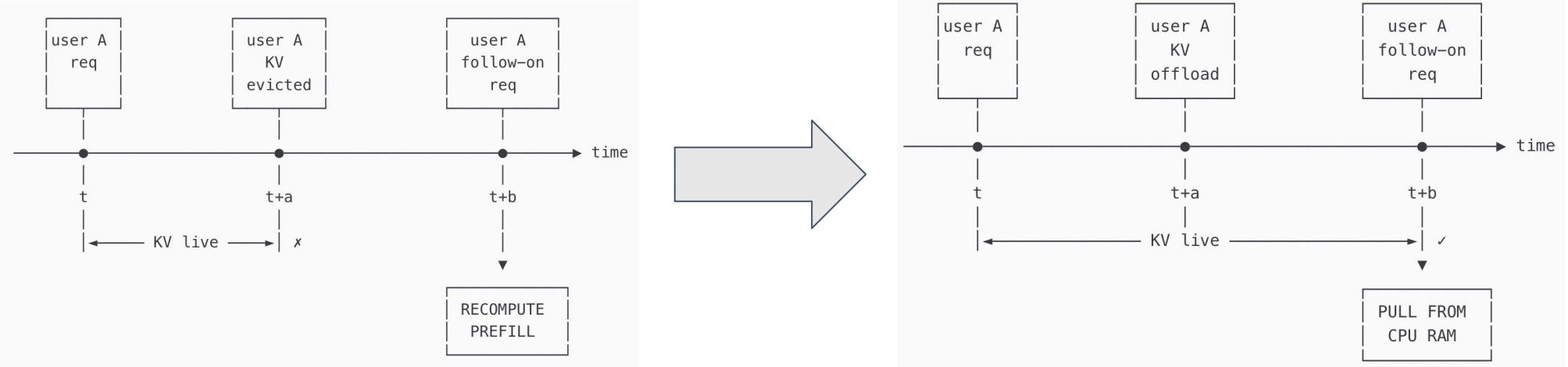


基于预测延迟的调度（Predicted Latency–Based Scheduling）

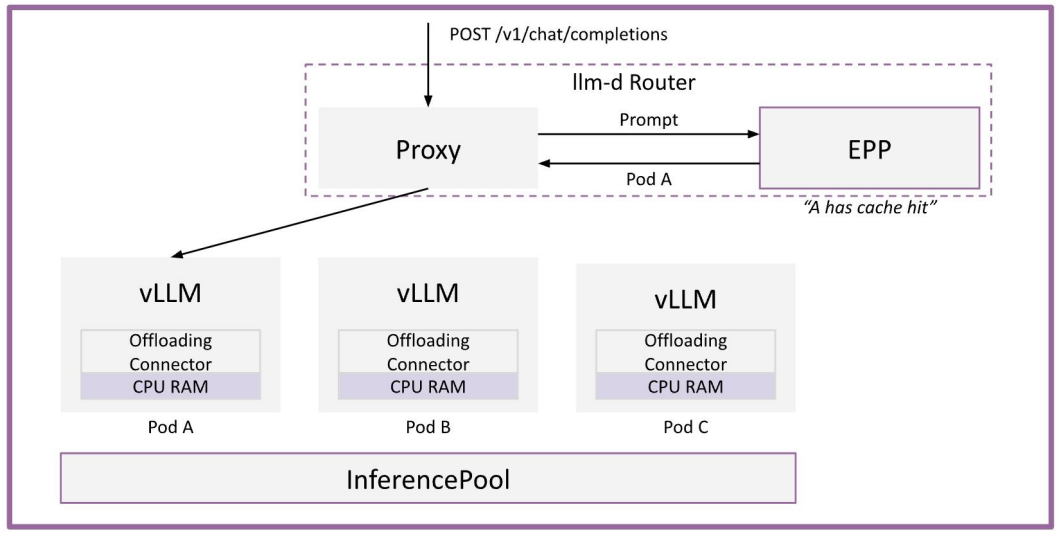



llm-d · Predicted Latency-Based Scheduling
Predict per-request latency for every candidate Pod, then route and learn from real outcomes
ONLINE REQUEST PATH · PREDICT → FILTER → SELECT → FORWARD
1
CLIENT / PROXY
Request arrives
Client sends an inference request to Gateway / Proxy / EPP.
2
EPP PRODUCER
Build feature vector
Read request tokens and live Pod state: queue, KV & prefix cache.
3
PREDICTION SERVER
Predict per Pod
Predict TTFT and, for streaming, TPOT for every candidate endpoint.
4
FILTERS
Narrow candidates
Apply prefix-cache affinity and SLO-headroom filters.
candidate set
SELECTION & FEEDBACK · ROUTE THE REQUEST, THEN TRAIN ON ACTUAL LATENCY
5
LATENCY SCORER
Score predicted latency
Minimize latency, or score by SLO headroom / deficit.
6
WEIGHTED PICKER
Select endpoint
Favor the best score, while spreading traffic probabilistically.
7
PROXY → POD
Forward & serve
Proxy forwards to the chosen model server; response streams.
8
ONLINE TRAINING
Learn from actuals
Send observed TTFT / TPOT / ITL to XGBoost training server.
MODEL LOOP
Train → sync → predict with fresh traffic
Training server retrains XGBoost on completed requests · prediction servers pull the latest model · fallback: heuristic scoring

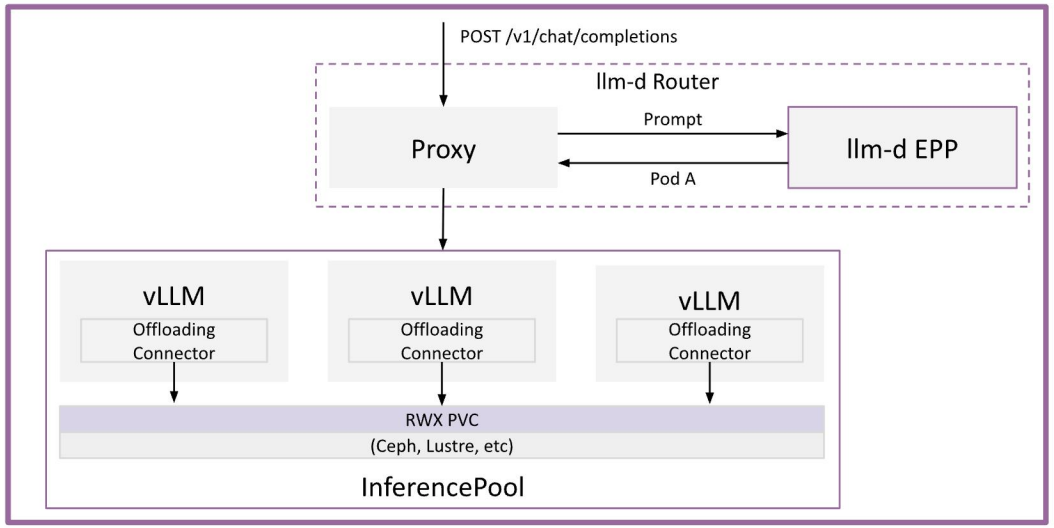
KV 缓存卸载（KV Cache Offloading）



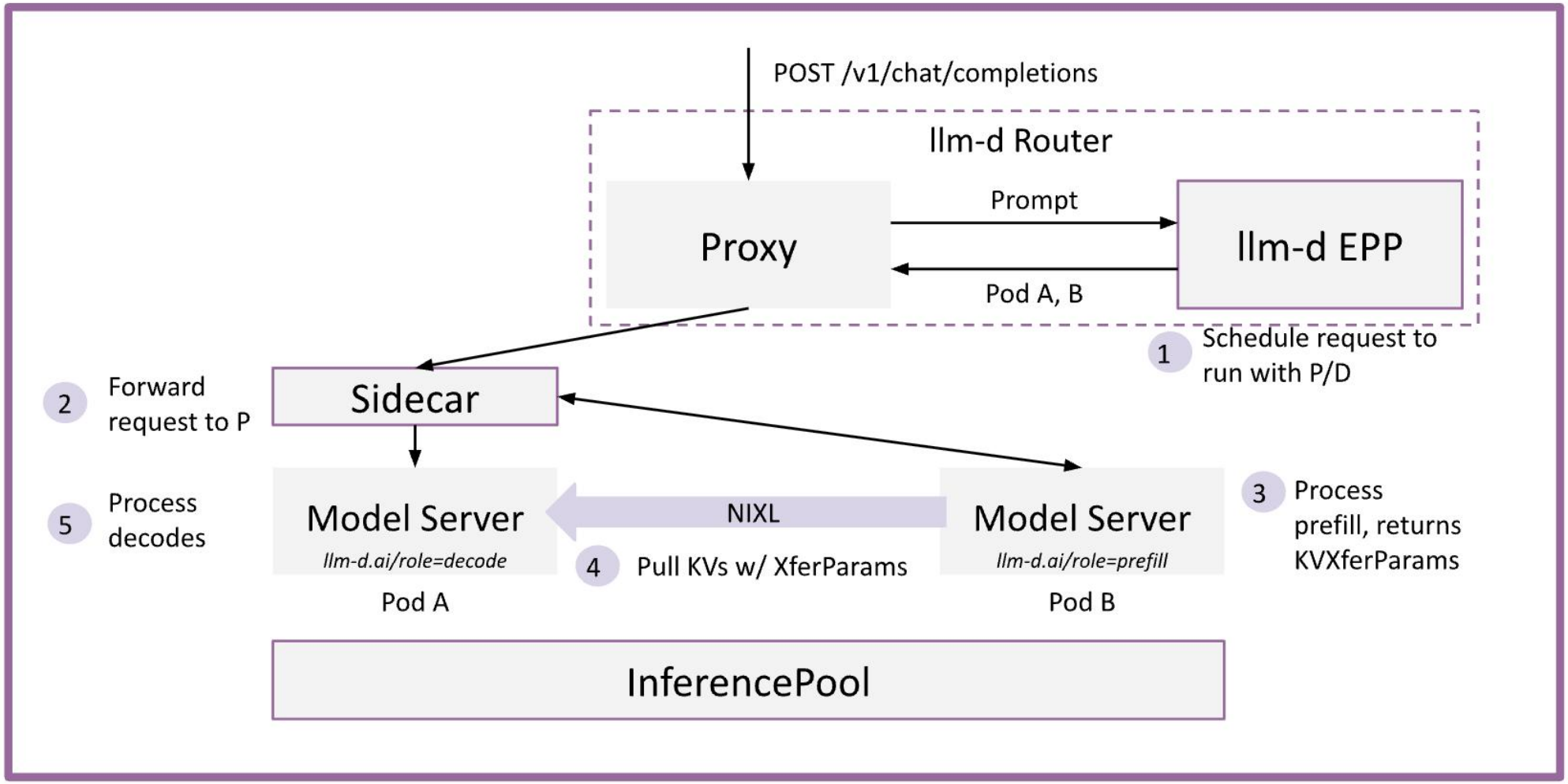
CPU KV 缓存卸载（CPU KV Cache Offloading）



分层卸载到文件系统（Tiered Offloading to Filesystem）

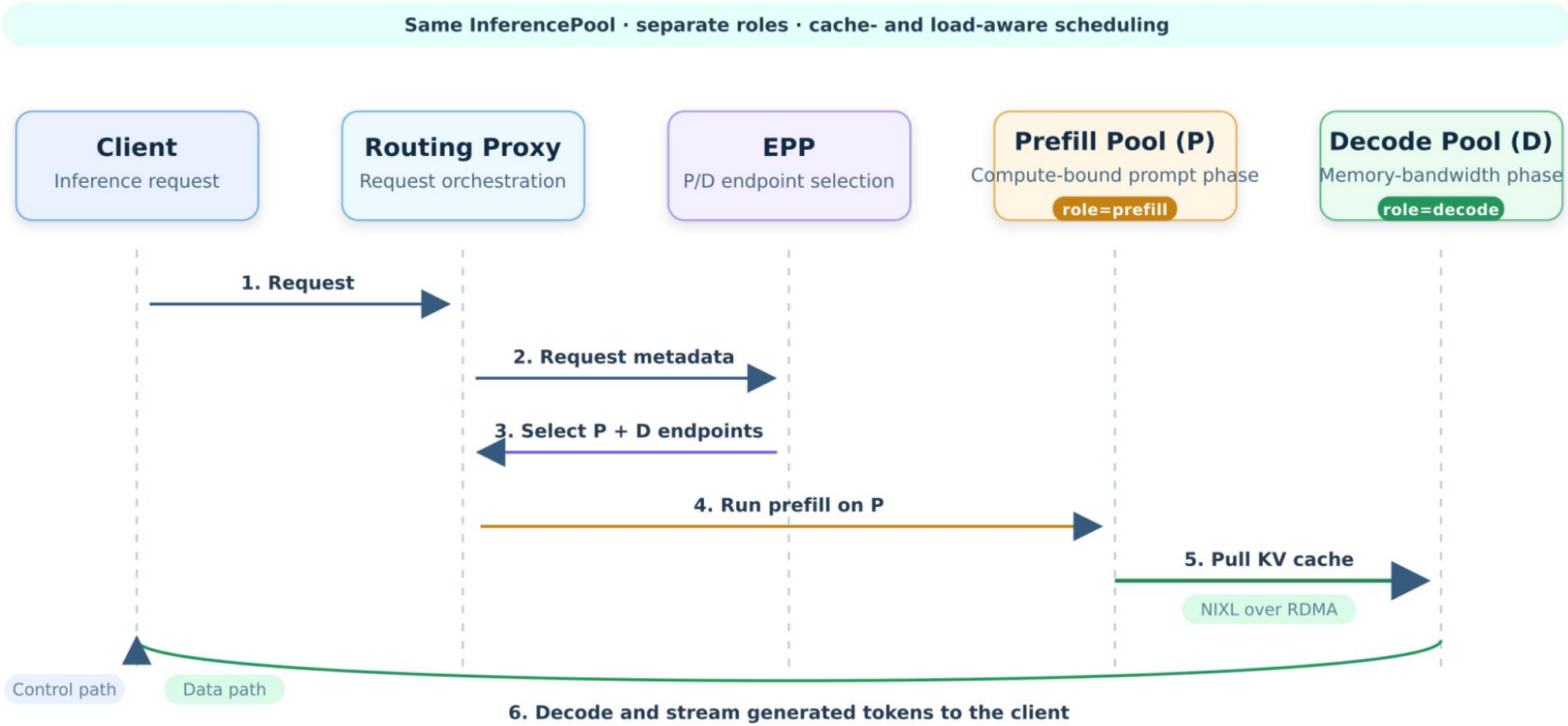


P/D 分离（Prefill/Decode Disaggregation）



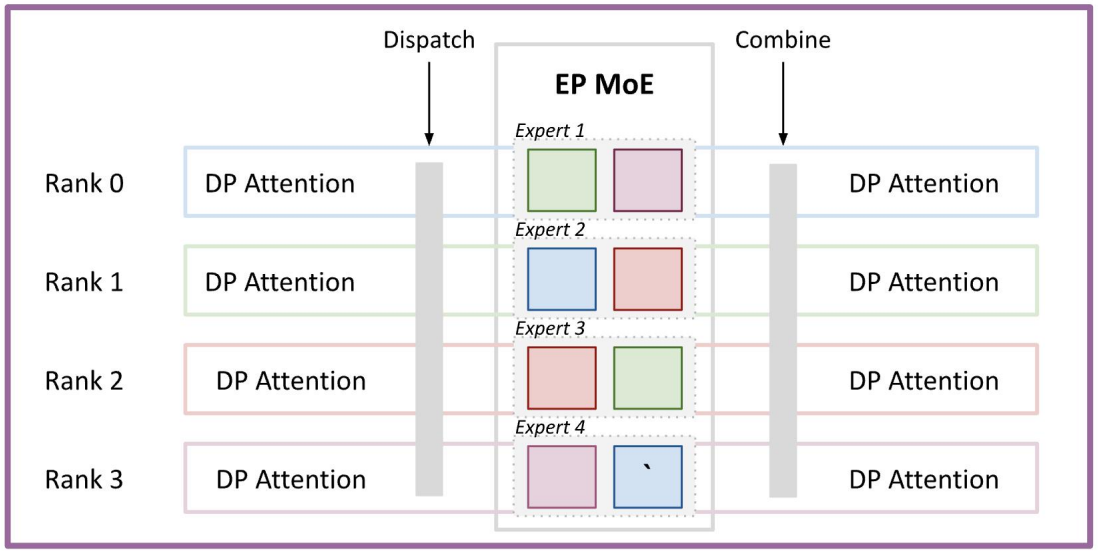
llm-d P/D Disaggregation — Request Flow

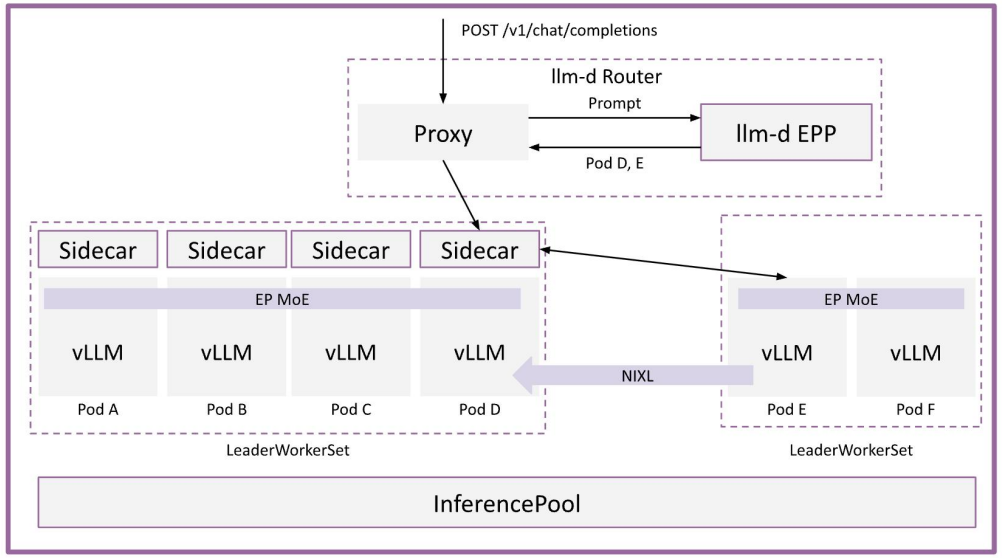
EPP selects specialized Prefill and Decode workers; KV cache moves directly from P to D.



多节点宽专家并行（Multi-Node Wide Expert Parallelism）

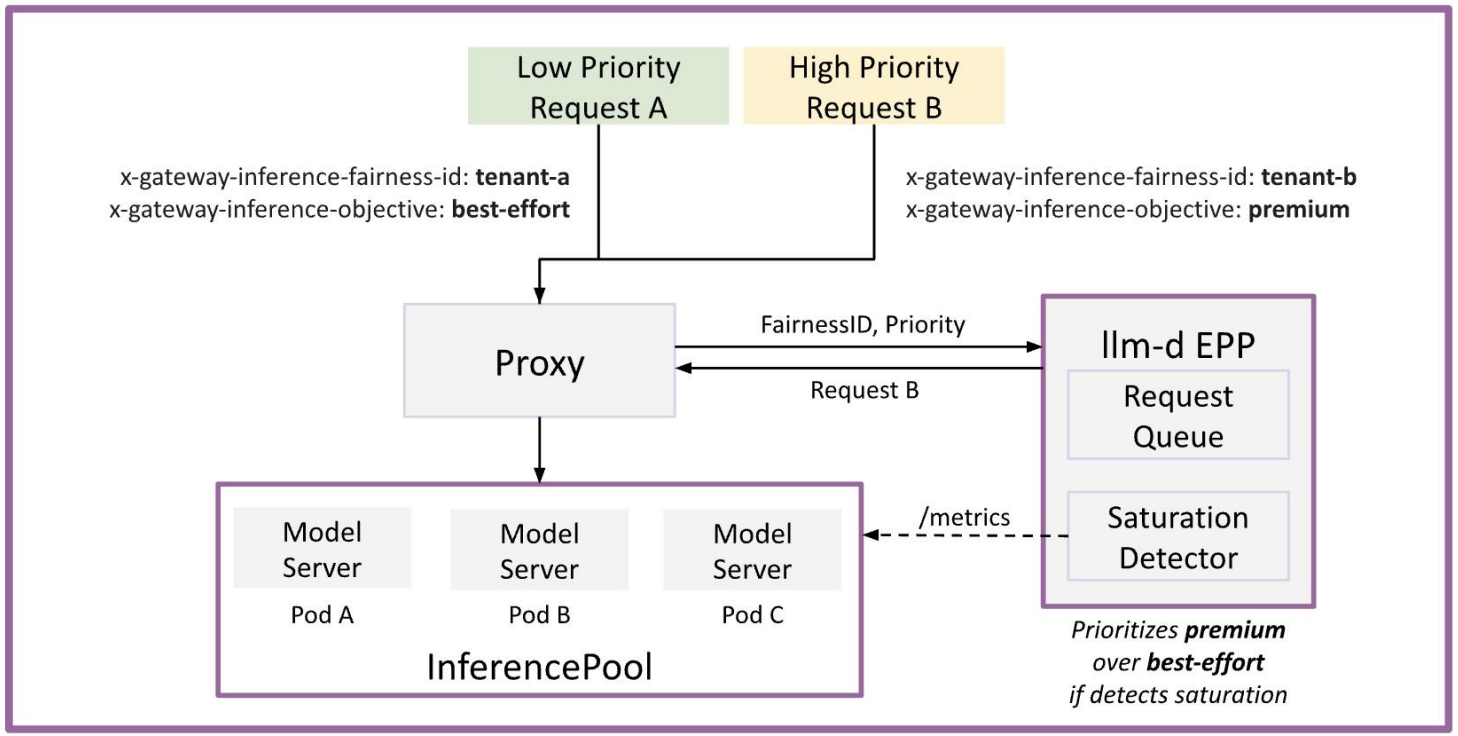
- **可扩展到多个节点：**由于集合通信操作（分发 / 汇聚）是稀疏的，Token 在路由后仅会发送给对应的专家进程（expert rank）。相比张量并行（TP）配置中使用的 All-Reduce，这种方式消耗的带宽更少，因此适合运行在速度较慢的互连网络上，例如 InfiniBand（IB）或 RoCE，而非 NVLink。
- **避免 KV 缓存副本：**由于注意力计算采用数据并行方式（每个数据并行组中的 TP=1），每个 Token 的 KV 缓存仅保留一份。
- WideEP 将 MoE 模型中的不同专家分布到多个节点上，**每个节点负责一部分专家的计算**。这种方式能够充分利用集群中所有节点的计算资源，同时**减少每个节点需要承载的专家数量，从而降低显存压力**。





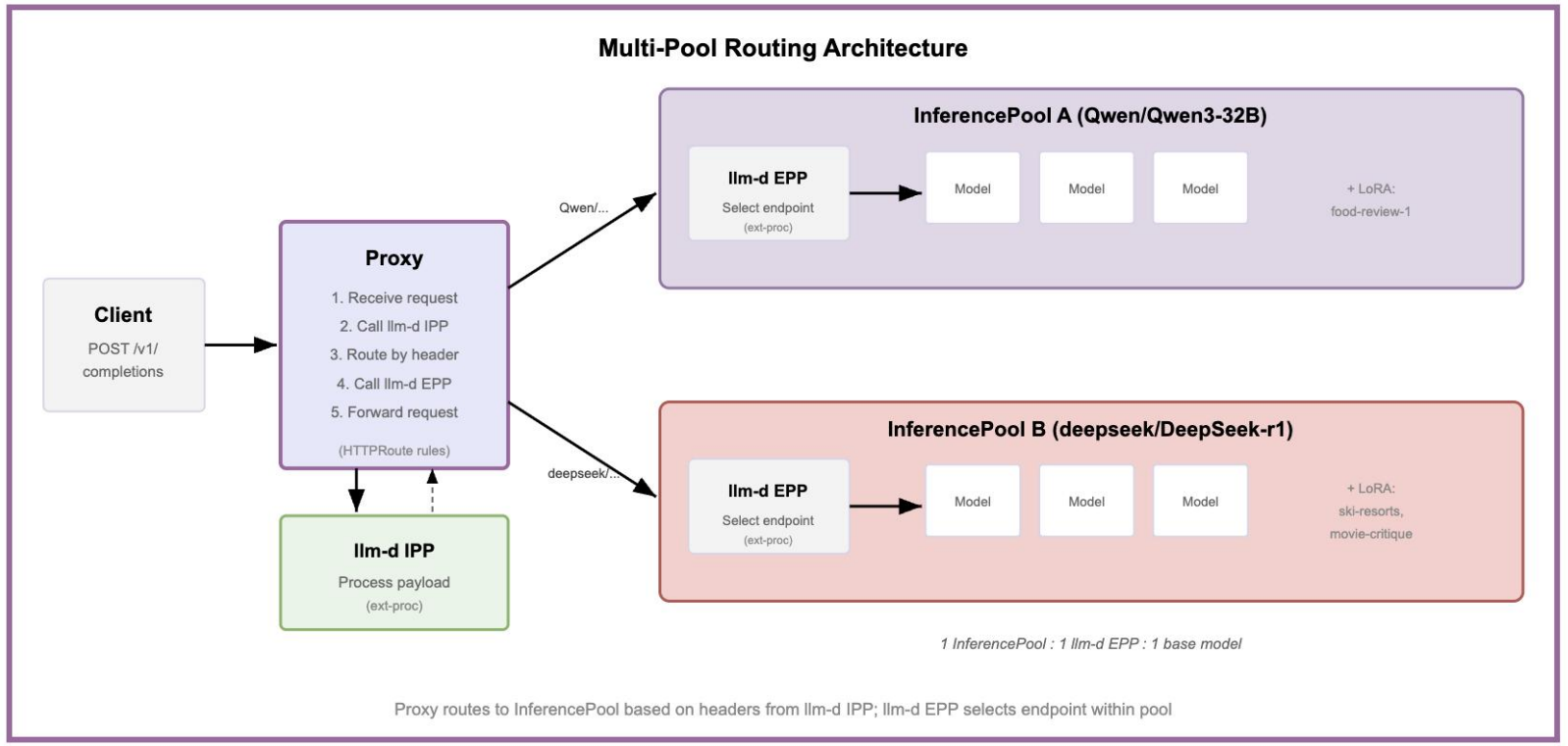
流量控制（Flow Control）

- 某些租户的**优先级更高**（例如：付费用户与免费用户）。
- 某些请求具有**不同的服务等级目标（SLO）**（例如：批处理请求与在线请求）。
- 某些租户比其他租户更活跃，因此需要在不同租户之间保证**公平性**。



多模型路由（Multi-Model Routing）

- 当这些模型共用同一个 API 路径（`/v1/chat/completions`）时，传统的基于路径的路由无法区分它们。
- **推理载荷处理器（Inference Payload Processor，IPP）**通过从请求正文中提取模型名称，并将请求路由至相应的 InferencePool（推理池），解决了这一问题。



工作负载自动扩缩容（Workload Autoscaling）

- 有效的 **LLM 自动扩缩容**需要主动、具备 SLO 感知能力的信号，以反映推理系统的真实状态——包括队列深度、飞行中请求数量和 KV 缓存压力——从而在最终用户延迟受到影响之前提前增加容量。
- llm-d 技术栈为**工作负载自动扩缩容**提供两条主要路径：
  - **EPP 路径**：使用由 KEDA 创建和管理的 HPA。
  - **WVA 路径**：发布期望副本数信号；根据部署配置，该信号可由 HPA 或 KEDA 使用。

KEDA + EPP 指标

此路径使用端点选择器（Endpoint Picker，EPP）直接输出的信号来配置 KEDA 的 Prometheus 扩缩容器。KEDA 生成的 HPA 利用队列深度、运行中请求数等指标，对排队情况和活跃请求并发度作出响应；当需求下降时再缩减副本数。

- **最适合**：硬件同构的部署环境；每个目标模型服务池都可以依据指标隔离，并独立扩缩容。
- **信号**：EPP 指标，例如队列深度、运行中请求数。
- **组件**：KEDA 和 Prometheus；不需要 Prometheus Adapter。

工作负载变体自动扩缩容器（WVA）

工作负载变体自动扩缩容器（Workload Variant Autoscaler，WVA）面向在共享且可能异构的 GPU 硬件上运行多个模型变体的运维人员。它持续监控 KV 缓存利用率、队列深度和性能预算，以确定不同变体的最优副本数，例如 A100 与 L4 上的模型实例。

- **最适合**：需要基于成本感知进行容量分配的多模型变体部署。
- **信号**：KV 缓存利用率、队列深度、能耗/性能预算。
- **组件**：WVA 控制器和 KEDA。WVA 会为带有 `llm-d.ai/managed: "true"` 注解的 KEDA `ScaledObject`（或 HPA）发布 `wva_desired_replicas` 指标，KEDA 据此执行扩缩容。自 llm-d 0.8.0 起，`VariantAutoscaling` CRD 已弃用。

快速模型激活（Fast Model Actuation）

- 快速模型激活（Fast Model Actuation，FMA）通过两项互补技术缩短 vLLM 启动时间：
  - **vLLM 休眠/唤醒：**将已加载模型的张量保留在主机内存中，数秒内恢复到 GPU；完全跳过模型加载和 CUDA Graph 编译。
  - **启动器（Launcher）进程：**只加载一次 vLLM Python 模块，并按需创建子 vLLM 实例，省去重复导入模块的开销。
- 两者结合，可将模型切换和副本扩容从分钟级冷启动缩短为近乎即时的操作，同时不影响稳定运行时的推理服务性能。

架构

FMA 通过由 FMA 控制器协调的双 Pod设计，避免 GPU 被独占并长期分配：

- **请求 GPU 的服务端 Pod：**通过 Kubernetes 调度器和 kubelet 预留 GPU 资源，以保证集群容量核算准确；但该 Pod 本身不运行推理。
- **启动器 Pod（提供服务）：**运行 vLLM，但请求的 GPU 数量为零。它们可借助不计入调度器资源使用量的机制访问所在节点的全部 GPU；FMA 控制器再通过 `CUDA_VISIBLE_DEVICES` 将 vLLM 指向由请求 Pod 预留的特定 GPU。
- **FMA 控制器：**创建和删除启动器，将请求方绑定到启动器及其内部的 vLLM 实例，创建和删除这些 vLLM 实例，并编排休眠/唤醒流程，使固定的 GPU 池能够承载多个模型变体。

激活路径

当需要额外容量时，控制器选择一个启动器，并采用当前可用的最快路径：

- **热启动：**通过 PCIe 将权重从 CPU 内存移回 GPU，唤醒一个已常驻但处于休眠状态的实例；耗时不足一秒。
- **温启动：**已有启动器利用预加载的模块创建新的 vLLM 实例；耗时数十秒。
- **冷启动（使用启动器）：**创建新的启动器 Pod，然后初始化实例；仍快于不使用 FMA 的冷启动，后者还需要承担 Pod 调度和镜像拉取的额外耗时。

为限制内存占用，控制器为每块加速器保留的休眠实例数量设有可配置上限；超过该预算时，会淘汰最近最少使用的实例。

服务多种类型的工作负载（Serve Different Types of Workloads）

智能体工作负载

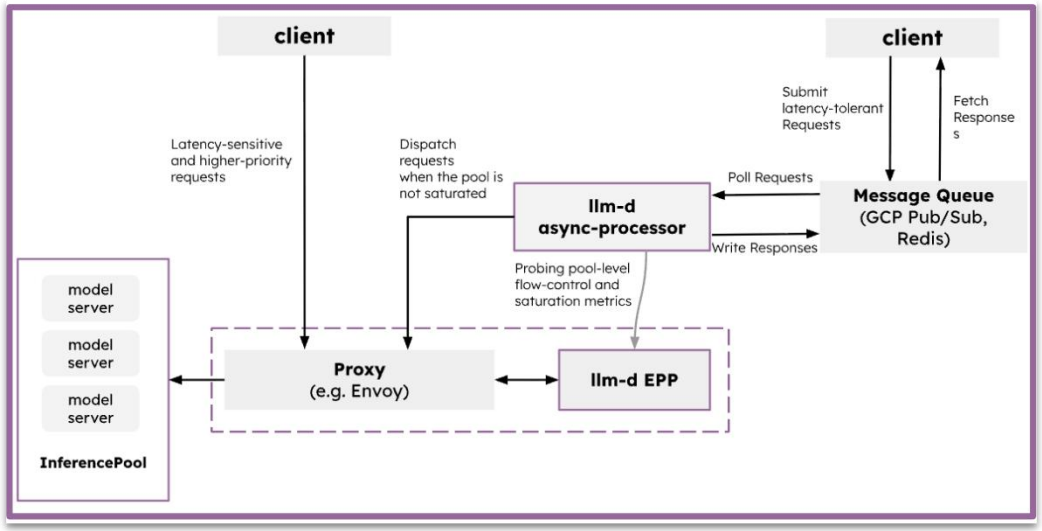
- **大量上下文复用：**推理树的分支、工具调用循环中的各轮次，以及智能体群（swarm）的子智能体，会共享大部分上下文，包括系统提示词、工具定义和此前的对话内容。
- **程序级目标：**用户关注的是整个程序的完成时间，而不是某一次调用的延迟；任意一个分支发生阻塞，都可能拖慢整个程序。
- **具备类型且生命周期可预测：**持久存在的系统提示词和工具，与短暂的草稿式思考内容不同；其执行流程本应可被技术栈提前预判，而不只是被动响应。

多模态工作负载

- **图像：**当图像以内嵌形式（原始字节）放在请求中时，EPP 会直接从图像内容读取宽度和高度。当图像以 URL 提供时，EPP 不会获取远程内容，而是回退到配置的默认值。
- **视频：**EPP 无法直接检查视频内容，因此客户端应通过请求头提供视频元数据。未提供的任何请求头都会回退到相应的 EPP 默认值：
  - `x-llm-d-video-fps`：视频每秒帧数。
  - `x-llm-d-video-duration-seconds`：视频时长，单位为秒。
  - `x-llm-d-video-resolution`：视频帧分辨率，格式为 `宽度x高度`，例如 `1280x720`。

批处理工作负载

- **批处理网关（Batch Gateway）：**企业级、全托管、兼容 OpenAI 的批处理 API（`/v1/batches`、`/v1/files`）。最适合多租户环境：客户端需要规范的异步作业提交、文件存储、状态跟踪，以及交互式计算与批处理计算之间的严格隔离。
- **异步处理（Asynchronous Processing）：**轻量、低开销的队列分发机制，可使用 Redis 或 GCP Pub/Sub。最适合需要低复杂度后台任务处理的内部微服务架构，或通过动态分发门控来利用推理池中的“空闲”容量。



04 最新的版本进展



1、简介 —— v0.8 版本的三个核心方向

- **背景：**v0.7 实现了各项功能的可部署性；v0.8 则是这些投入的成果兑现
- **三大核心主题：**
  - **将 v0.7 的实验性功能转为生产就绪状态**
  - **扩展至 Kubernetes 之外的场景（如 RL 训练、Slurm）**
  - **重新定位 llm-d 的角色：**作为推理控制平面（inference control plane），而非推理引擎的分支（engine fork）
- **拓宽应用范围：**在同一部署中支持交互式聊天、批处理、多模态及智能体（Agent）工作流

2、兑现承诺：功能正式上线（从实验转为生产）

- **流量控制（Flow Control）→ 生产就绪：**基于 FlowKey 的分类、流级队列、优先级分层、租户公平性保障，以及高负载下的“无悔调度”（no-regret scheduling）
- **批处理网关（Batch Gateway）→ 生产就绪：**兼容 OpenAI 的 `/v1/batches` 和 `/v1/files` 接口，支持单任务高达 5 万次请求，支持可插拔存储（PostgreSQL/Redis/S3），并通过流量控制的背压机制与交互式流量共存
- **多模态服务 → 生产部署指南**（涵盖 LLaVA、Qwen-VL），包括解耦的预填充/解码（P/D）路径；模拟器支持多模态负载（payloads）



3、拥抱上游，而非分叉（Fork）：与 vLLM 对齐

- **默认镜像切换为未经修改的上游 vLLM 镜像**（vllm/vllm-openai:v0.23.0，涵盖 ROCm 和 XPU 版本）；移除了自定义的 llm-d-cuda-gb200/xpu/rocm 镜像
- **定位声明：**llm-d 的核心价值在于控制平面（包括路由、端点选择、KV 缓存层及自动扩缩容），而非单纯分发推理引擎
- **组件重命名以体现功能：**llm-d-inference-scheduler 更名为 llm-d-router-endpoint-picker；routing-sidecar 更名为 router-disagg-sidecar
- **停止支持 HPU（Intel Gaudi）**（因上游 vLLM 缺乏相应镜像）

4、超越 Kubernetes：RL、Slurm 与文件发现机制

- **FileDiscovery 插件：**引入基于文件系统的端点发现机制，不再依赖 K8s API
- **主要驱动场景：**强化学习（RL）训练循环——在 Slurm 或裸机环境下进行高吞吐量的 rollout 生成，并将调度器嵌入训练循环中
- **核心理念：**路由智能（如前缀缓存局部性、负载饱和度感知、流控机制）在任何推理场景下均具有价值

5、智能路由：赋能智能体（Agentic）时代

- **智能体工作负载路由与 Responses API：**支持在多步智能体链（agent chains）中追踪会话亲和性与前缀缓存状态
- **针对 DeepSeek 级 MoE 模型的 DP（数据并行）感知调度（结合 WideEP 技术）：**功能已成熟并纳入每日 CI（持续集成）流程
- **基于延迟预测的调度：**全面重写实现，提供生产环境部署指南，CI 流程验证实际预测结果与启发式回退策略的对比情况

6、多层级 KV 缓存与混合模型支持

- **支持 HMA（异构内存架构）感知的 KV 卸载：**针对混合架构（如 Jamba，结合了全注意力、滑动窗口、Mamba/SSM 及线性注意力机制）优化；根据异构内存布局，实现 HBM → DRAM → 存储的卸载流程
- **Mooncake 连接器：**将阿里巴巴的 KV 传输引擎作为 NIXL 后端，与 UCX 和 UCCL 共同构成三大传输引擎
- **存储事件：**基于容量或驱逐信号的响应式分层管理机制

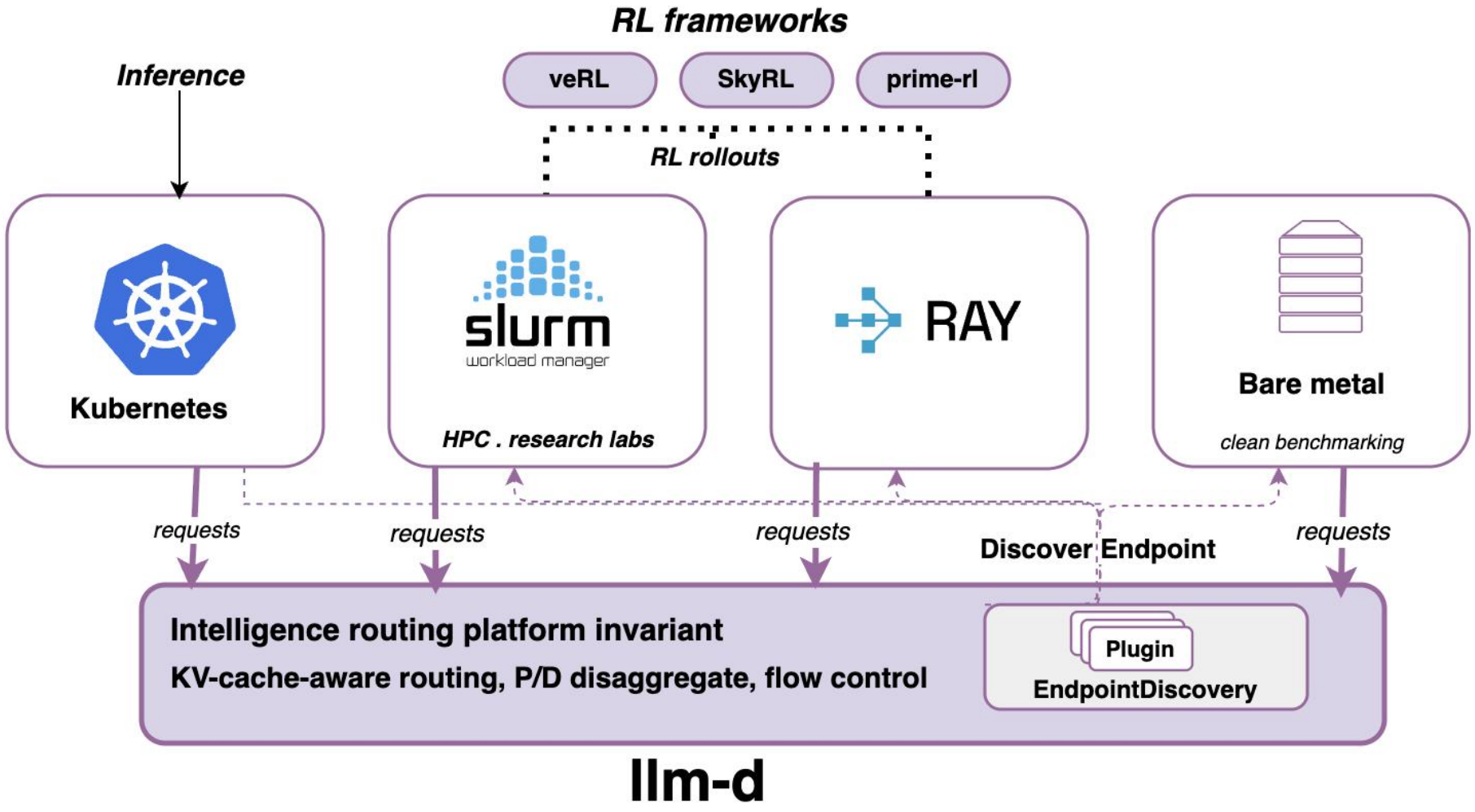
7、优化的应用路径与生态扩展

- **DeepSeek-V4 在 GB200 NVL72 上的部署：**提供包含 WideEP 和 NVSHMEM 配置的专项指南
- **Envoy AI Gateway 指南：**继 Istio 和 kgateway 之后的第三种网关方案
- **WVA 自动扩缩容 CRD** 迁移至 llm-d.ai API 组
- **更新基础设施依赖**（Istio 1.29.4、kgateway v2.3.3、GAIE v1.5.0）
- **P/D（预测/解码）解耦：**支持 GKE RDMA/DRA/DRANET 配置及网络预检功能

8、面向不同角色的价值

- **平台团队：**Flow Control（流控）与 Batch Gateway（批处理网关）功能正式发布；简化镜像构建流水线；WVA 迁移涉及破坏性变更
- **ML 研究人员：**支持在 Slurm 强化学习（RL）循环中实现感知缓存的路由调度（无需 Kubernetes）
- **Agent 架构师：**支持会话亲和性（session affinity）、Responses API 以及与底层引擎无关的 Agent 路由机制

超越 Kubernetes — llm-d 现在可以运行在任何地方

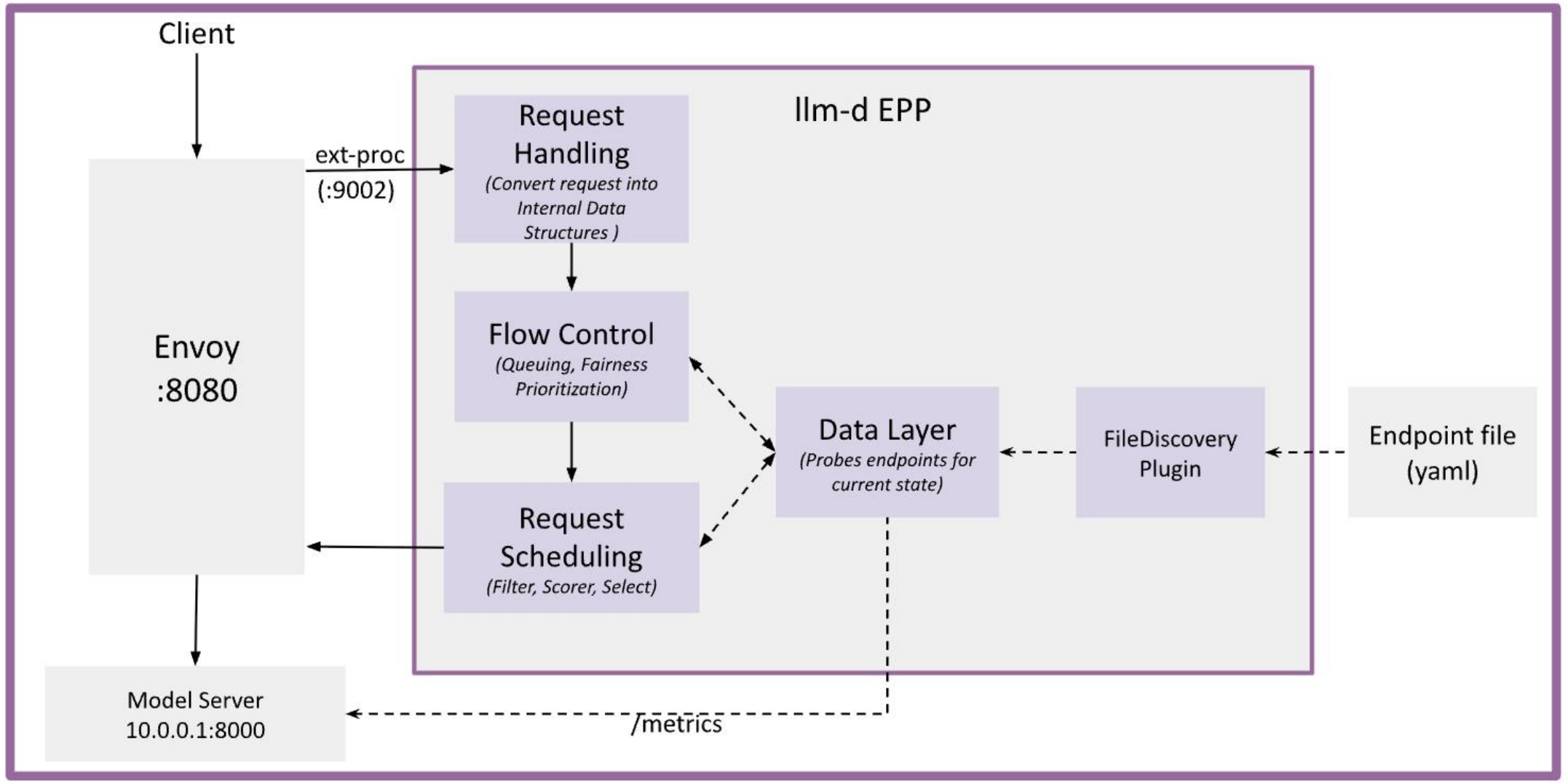


EndpointDiscovery 接口

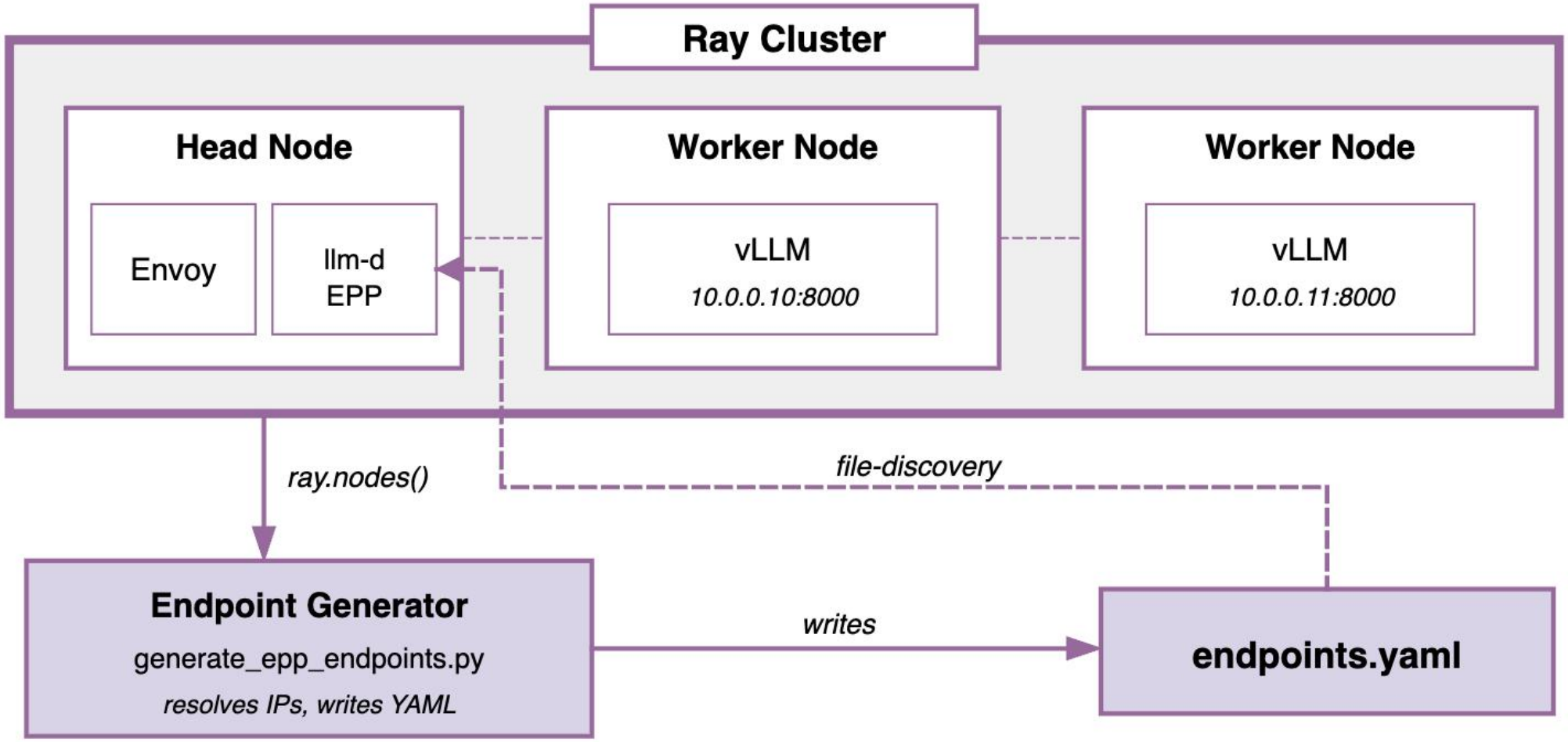

```
type EndpointDiscovery interface {
    fwkplugin.Plugin
    // Start begins discovery; blocks until ctx is cancelled or a fatal error occurs.
    Start(ctx context.Context, notifier DiscoveryNotifier)error
    // Ready is used to gate request-serving until the datastore is populated.
    Ready()<-chanstruct{}
}
type DiscoveryNotifier interface {
    // Upsert adds or updates an endpoint in the datastore.
    Upsert(endpoint *EndpointMetadata)
    // Delete removes an endpoint from the datastore.
    Delete(id types.NamespacedName)
}
```



EndpointDiscovery 架构示意图

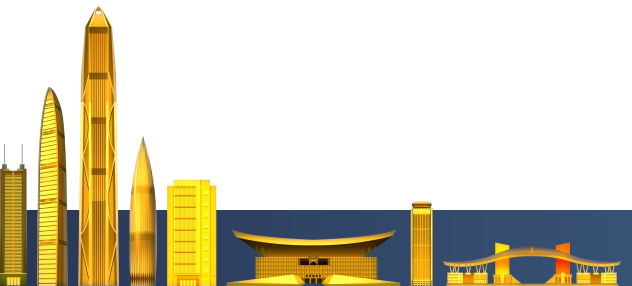


EndpointDiscovery 与 Ray Cluster 的集成



llm-d v0.9: 面向规模的加固（Hardened for Scale）

- 生产级加固路由器
- 解耦：生产级运维
- 路由智能
- 多模态与扩散模型支持
- KEDA 原生自动扩缩容
- 强化学习：孵化与发展方向



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!

